EXPERIMENT – 12

Study of Camera Phishing Using an Open-Source GitHub Tool in Kali Linux (Educational Demonstration)

Aim

To study the concept of camera phishing attacks using an open-source GitHub tool in Kali Linux in order to understand privacy threats and the importance of secure user awareness.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tool:** Open-source GitHub camera phishing framework (educational use)
- **System Requirements:**
    - RAM: Minimum 4 GB (8 GB recommended)
    - Disk Space: Minimum 30 GB
    - Processor: Intel/AMD with Virtualization support

Introduction

Camera phishing is a social-engineering–based attack where victims are tricked into granting camera access through malicious links or fake web pages. Such attacks can compromise user privacy and are commonly used in cybercrime, surveillance, and identity theft scenarios. Studying camera phishing in Kali Linux using open-source tools helps students understand how attackers exploit human behavior rather than software vulnerabilities. This experiment focuses on **awareness, detection, and prevention**, not real-world exploitation.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Open-Source Tool

1. Access an open-source GitHub repository related to camera phishing (educational purpose).
2. Review the documentation to understand:
    - How fake permission prompts are created
    - How user interaction is exploited
    - Ethical limitations of such tools

Step 3: Understand the Attack Workflow

1. Analyze how phishing pages request camera permissions.
2. Observe how browsers handle camera access requests.
3. Study how attackers misuse user trust.

Step 4: Simulated Demonstration

1. Perform a **local, consent-based simulation** using test accounts.
2. Observe how camera permission prompts appear.
3. Analyze user response behavior.

Step 5: Analyze Security Implications

1. Identify risks related to:
    - Unauthorized camera access
    - Privacy leakage
    - Social engineering
2. Discuss how modern browsers and OS security attempt to mitigate such attacks.

Step 6: Study Prevention Techniques

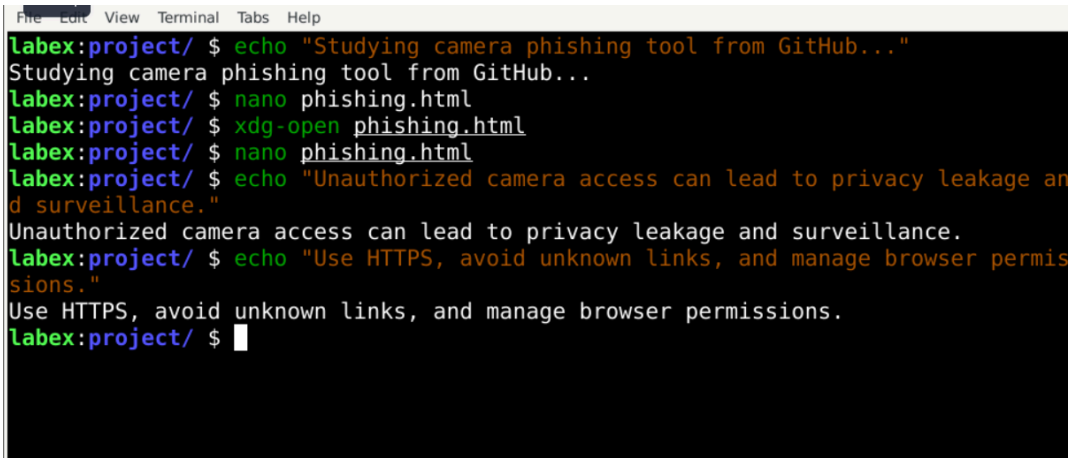
- Browser permission management
- User awareness and training
- HTTPS and trusted domains
- OS-level camera access controls

Output

```
  GNU nano 6.2                    phishing.html
<!DOCTYPE html>
<html>
<head>
<title>Camera Access</title>
</head>
<body>
<h2>Allow Camera Access</h2>
<button onclick="startCamera()">Allow Camera</button>

<video id="video" width="300" autoplay></video>

<script>
function startCamera() {
navigator.mediaDevices.getUserMedia({ video: true })
.then(stream => {
document.getElementById('video').srcObject = stream;
})
.catch(err => {
alert("Permission denied");
});
```



Result

The study of camera phishing using an open-source GitHub tool was successfully completed in Kali Linux.
The experiment enhanced understanding of social-engineering attacks and emphasized the importance of user awareness and privacy protection.